Observability & Logging Standards

Engineering Handbook · Internal · Owner: Platform Engineering

Purpose

This page sets the logging, tracing, metrics, and health-check standards for every service on the platform (api-service, auth-service, worker-service, ingestion-service, gateway, scheduler). Good observability lets us join a request across services, keep signal-to-noise high, control cost, and never leak sensitive data. These rules apply in every environment; production adds stricter defaults for verbosity and redaction.

Standards

Structured logging and levels

Emit structured (JSON) logs so fields are queryable and volume is controllable. The log level comes from configuration, not code — default to INFO or higher in production. Reserve DEBUG for local and short-lived diagnostic sessions, and rate-limit or sample noisy loggers.

```
import logging, os, json
class JsonFormatter(logging.Formatter):
   def format(self, r):
       return json.dumps({
           "severity": r.levelname, "message": r.getMessage(),
           "logger": r.name, "trace": getattr(r, "trace_id", None),
       })
level = os.environ.get("LOG_LEVEL", "INFO").upper()  # INFO+ in prod, never hard-coded DEBUG
logging.basicConfig(level=level, handlers=[_h_with(JsonFormatter())])
```

Redaction — never log PII or secrets

Never log full request/response bodies, auth tokens, or credentials. Redact known-sensitive fields at the boundary with a blocklist and a scrubbing filter so accidental payload logging cannot leak data.

```
_SENSITIVE = {"password", "token", "authorization", "email", "ssn"}
def redact(d: dict) -> dict:
   return {k: ("***" if k.lower() in _SENSITIVE else v) for k, v in d.items()}
```

- Log identifiers (user id hash, order id), not raw personal data.
- Add a logging filter that scrubs known PII patterns (emails, bearer tokens) as a backstop.

Trace correlation, metrics, and health

- Propagate a W3C `traceparent` correlation id through every inbound and outbound call, and include it on every log line so cross-service logs can be joined.
- Keep metric labels low-cardinality. Never use user id, request id, or raw path as a label; bucket values and use exemplars/traces for per-request detail.

- Separate liveness from readiness. Readiness must verify critical dependencies (DB, cache) with short timeouts so a degraded instance drains instead of serving traffic.

```
@app.get("/readyz")
async def readyz():
   await asyncio.wait_for(db.ping(), timeout=0.5)
   await asyncio.wait_for(cache.ping(), timeout=0.2)
   return {"status": "ready"}
```

Common failures and fixes

- Logging cost and disk usage spike from DEBUG left on in prod → drive log level from config (INFO+), sample noisy loggers, emit structured logs (BUG-1071).
- Emails/tokens appear in logs from logging bodies verbatim → redact/blocklist sensitive fields, never log tokens or full bodies, add a scrubbing filter (BUG-1072).
- A cross-service failure can't be traced → propagate a W3C traceparent correlation id and include it in every log line (BUG-1073).
- Metrics backend degrades and costs spike → remove unbounded labels (user/request id), bucket values, keep label sets low-cardinality (BUG-1074).
- Load balancer keeps traffic on a broken instance → add a readiness check that verifies DB/cache with timeouts, separate from liveness (BUG-1075).

Checklist

- Logs are structured JSON with a severity field.
- Log level is read from config; production defaults to INFO or higher.
- No bodies, tokens, or PII logged; a redaction filter is installed.
- A traceparent correlation id is propagated and logged on every line.
- Metrics use only low-cardinality labels.
- Readiness probe checks real dependencies with timeouts, distinct from liveness.

Related incidents

BUG-1071, BUG-1072, BUG-1073, BUG-1074, BUG-1075